Chapter 6
Object-Oriented System Design

From High-Level Architecture to
Detailed Specification

Lecture Outline

1. **Introduction:** Object Oriented System Design
2. **Part I: System Design**
   - Overview, Concepts, Activities, and Documentation
3. **Part II: Object Design**
   - Overview, Concepts, Activities, and Management
4. **Conclusion:** Documentation

Introduction

What is Object Oriented System Design?

- **The Big Picture:** It is the bridge between **Requirements Analysis** (what the system should do) and **Implementation** (writing the code).
- **The Shift:** We move from "Problem Domain" objects to "Solution Domain" objects.
- **Goal:** To define the system architecture and the detailed behavior of the objects that will implement the requirements.

An Overview of System Design

Definition: System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements.

Focus Areas:

- Defining the **Design Goals** (e.g., performance vs. maintainability).
- Decomposing the system into smaller, manageable subsystems.
- Developing strategies for hardware/software mapping.

- **Key Shift:** The focus shifts from the "Problem Domain" (user concepts) to the "Solution Domain" (technical concepts like servers, databases, and protocols).
- System design is about the *structure* of the solution, not the code details yet.
- **Input**: The "Analysis Model" (Use Cases, Domain Model).
- **Output**: The "System Architecture" (Subsystems, hardware mapping).

Example:

- **Analysis:** We know the system needs a "Register for Course" use case.
- **System Design:** We decide this system will be a **Web-Based Application** consisting of a Frontend (React), a Backend API (Java/Spring), and a Database (PostgreSQL). We are defining the *architecture*.

System Design Concepts

Decomposition: a replaceable part of the system with well-defined interfaces that encapsulates the state and behavior of its contained classes is **Subsystem**.

Analogy: In a car, the "Engine" is a subsystem. It has many parts inside (pistons, spark plugs), but to the rest of the car, it is just a unit that provides power.

Coupling & Cohesion:

- **High Cohesion:** Classes within a subsystem are closely related.
- **Low Coupling:** Dependencies between different subsystems are minimized.

Example:

- **High Cohesion:** The BillingSubsystem contains Invoice, Payment, and Receipt. These all strictly belong together.
- **Low Coupling:** The CourseCatalog subsystem doesn't need to know *how* the BillingSubsystem processes a credit card; it just sends a request to "Process Payment."

Architectural Styles: Choosing a pattern (e.g., Client-Server, Peer-to-Peer, MVC, Layered Architecture)

- Client/Server
- Peer-to-Peer
- Model-View-Controller (MVC): Separates data (Model) from presentation (View) and logic (Controller).

Hardware Mapping: Deciding which software components run on which hardware nodes (Deployment).

System Design Activities: From Objects to Subsystems

This phase involves transforming analysis objects into a structured architecture through specific steps:

1. Identify Subsystems: Group analysis classes into subsystems based on functionality (e.g., User Management, Payment Processing).

2. Map Subsystems to Hardware: Decide which components run on which server or device Using **Deployment Diagrams** to show nodes (servers, devices) and artifacts (software).

cont..

3. Define Data Management: Choose between flat files, relational databases, or object databases.
4. Define Access Control: Determine who can access which objects.
5. Define Control Flow: How does the system sequence operations? (e.g., event-driven vs. procedure-driven).

Example (Subsystem Decomposition): We split the University System into distinct modules:

- **Authentication Subsystem:** Handles Login/Logout.
- **Course Catalog Subsystem:** Manages Course and Prerequisite data.
- **Student Records Subsystem:** Manages Grades and Transcripts.
- **Billing Subsystem:** Connects to external banks.
- *Note:* We use a **UML Component Diagram** to visualize this.

Documenting System Design

The **System Design Document (SDD)** captures the architectural decisions.

- **Purpose:** To communicate the architecture to stakeholders and developers.
- **Design Goals:** Explicitly lists trade-offs (e.g., "Reliability is more important than Performance").
- **Subsystem Decomposition:** Diagrams showing the subsystems and their dependencies.
- **Hardware/Software Mapping:** Describes the physical environment (Servers, Networks).
- **Data Management:** Describes the Database schema designs

Example:

- **Design Goal Decision:** "We prioritize **security** over **speed** for the Billing Subsystem."
- **Data Management Decision:** "We will use a Relational Database (SQL) for Student Records because data integrity is critical, but we will use a NoSQL cache (Redis) for the Course Catalog to make searching faster."

Object Design

Definition: Object design is the process of adding implementation details to the system design. It determines the full definitions of classes and associations.

- **Goal:** To define the solution domain objects that bridge the gap between the analysis model and the hardware/software platform.
- **Key Action:** We refine the "Analysis Objects" and introduce "Solution Objects" (like helper classes, UI widgets, database connectors) that don't exist in the real world but are needed for coding.

Object Design Concepts

Refining the Model

Visibility:

- Deciding who can see attributes and methods.
- **Private (-):** Only the class itself.
- **Public (+):** Everyone.
- **Protected (#):** Subclasses only.

Type Signatures: Defining exact data types (e.g., int, string) for all operations.

Polymorphism & Inheritance: Identifying where to use abstract classes and interfaces to simplify code.

Example (Refining the "Course" Class):

- ***Attribute:*** capacity becomes private int capacity.
- ***Method:*** checkAvailability() becomes public boolean checkAvailability().
- ***Optimization:*** Instead of calculating the number of enrolled students every time we check capacity, we add a new attribute currentEnrollmentCount to cache that number.

Object Design Activities

Detailed Specification

- **Reuse:** Check for existing libraries, frameworks, or design patterns (e.g., Singleton, Factory, Observer) to avoid writing custom code.
- **Interface Specification:** Define the exact API for each class. This includes method names, parameters, return types, and exceptions.
- **Restructuring:** Refactor the design to improve readability and extensibility.
- **Optimization:** Tweaking the design for speed or memory usage based on the design goals set in System Design.

Managing Object Design

Keeping the Team on Track

- **Interface Management:** Because many developers work in parallel, changing an interface (e.g., renaming a function) can break someone else's code. Management involves strict version control and communication of API changes.
- **Buy vs. Build:** Deciding whether to program a complex component from scratch or buy a commercial component (COTS - Commercial Off-The-Shelf).
- **Collaboration:** Using Unified Modeling Language (UML) as the common language for the team.

Example:

- The **System Architect** defines the interface: registerStudent(int studentID, int courseID).
- **Developer A** writes the code inside Student to request the registration.
- **Developer B** writes the code inside Course to accept the registration.
- Because the interface was agreed upon first, their code works together perfectly.

Documenting Object Design

- The **Object Design Document (ODD)** is the blueprint for programmers.
  - **Detailed Class Diagrams:** Includes all attributes, methods, visibility, and types.
  - **Sequence Diagrams:** visualizes how objects interact over time to fulfill specific use cases.
  - **Constraint Specifications:** Textual descriptions of business rules and invariants (often using OCL - Object Constraint Language).

Role: This document serves as the primary blueprint for programmers.

Example:

- **Constraint:** "The calculateGPA() method must return a value between 0.0 and 4.0. If no courses are taken, return -1.0."
- **Diagram:** A detailed Sequence Diagram showing exactly how DatabaseConnection throws an exception if the network is down.

Practical Activity: on Case Study

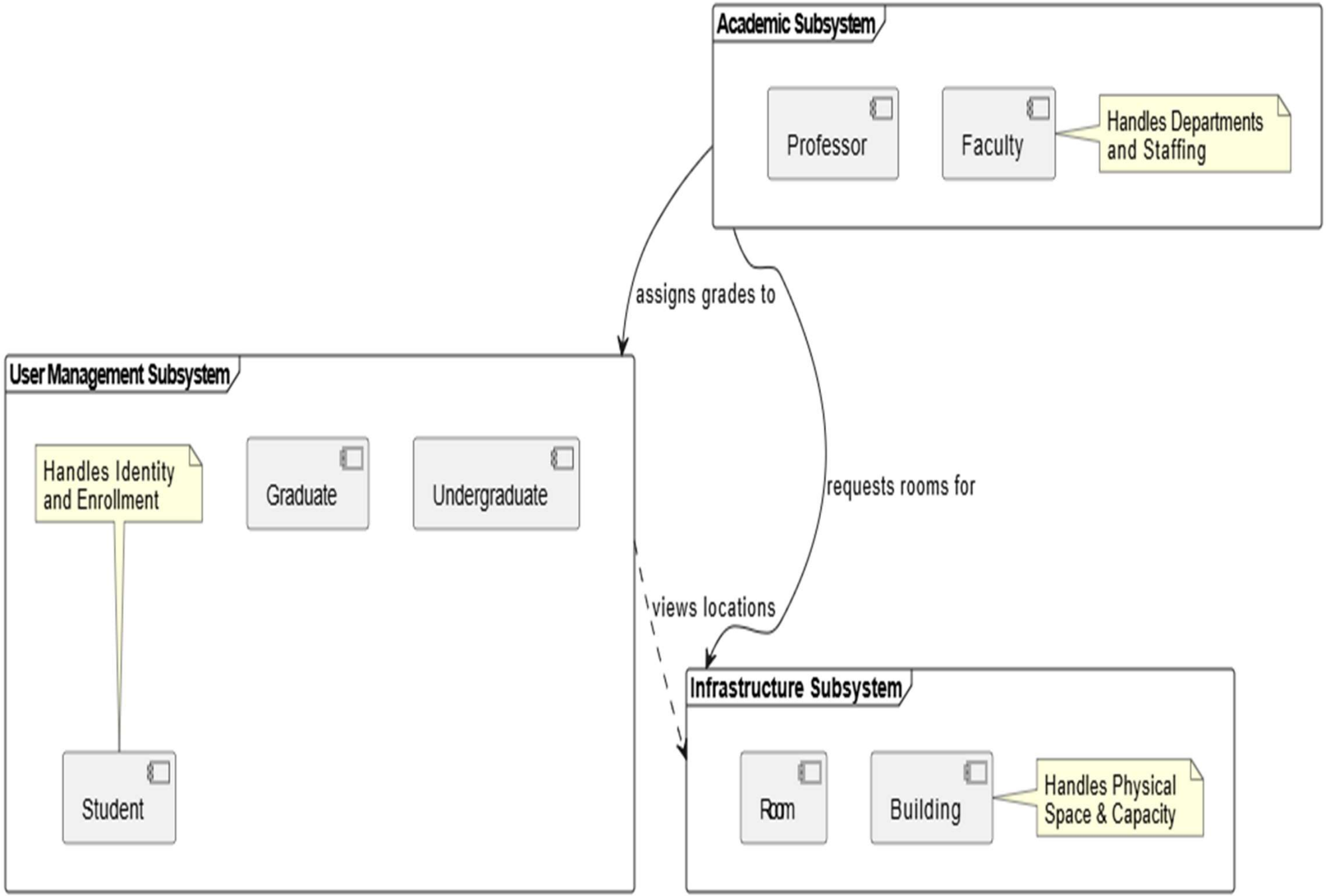
Directions:

- **Start with your Analysis Diagram:** This is your "Source of Truth." Do not delete it; you are evolving it.
- **Use "Design Stereotypes":** Label your new design classes as <<boundary>>, <<control>>, or <<entity>> to show their role in the software.
- **Think like a Compiler:** If you were the computer, would you have enough information to run this? (e.g., If a method returns a "Grade," is that a Letter or a Number?)
- **Traceability Check:** For every new method you add in Design, point back to the Requirement in the Case Study that requested it.

The University System Transformation

- **Phase 1: System Design (Architecture)**
- *Focus: High-level organization and decision making.*
- **Step 1: Subsystem Decomposition**
- **Task:** Look at your Analysis Class Diagram. Group the classes into three logical **Subsystems**: *Academic*, *Infrastructure*, and *UserManagement*.
- **Skill:** Understanding **Cohesion**. Why do Building and Room belong together but Professor and Room do not?
- **Output:** A simple block diagram (Component Diagram) showing the three subsystems and the lines (interfaces) connecting them. See next slide

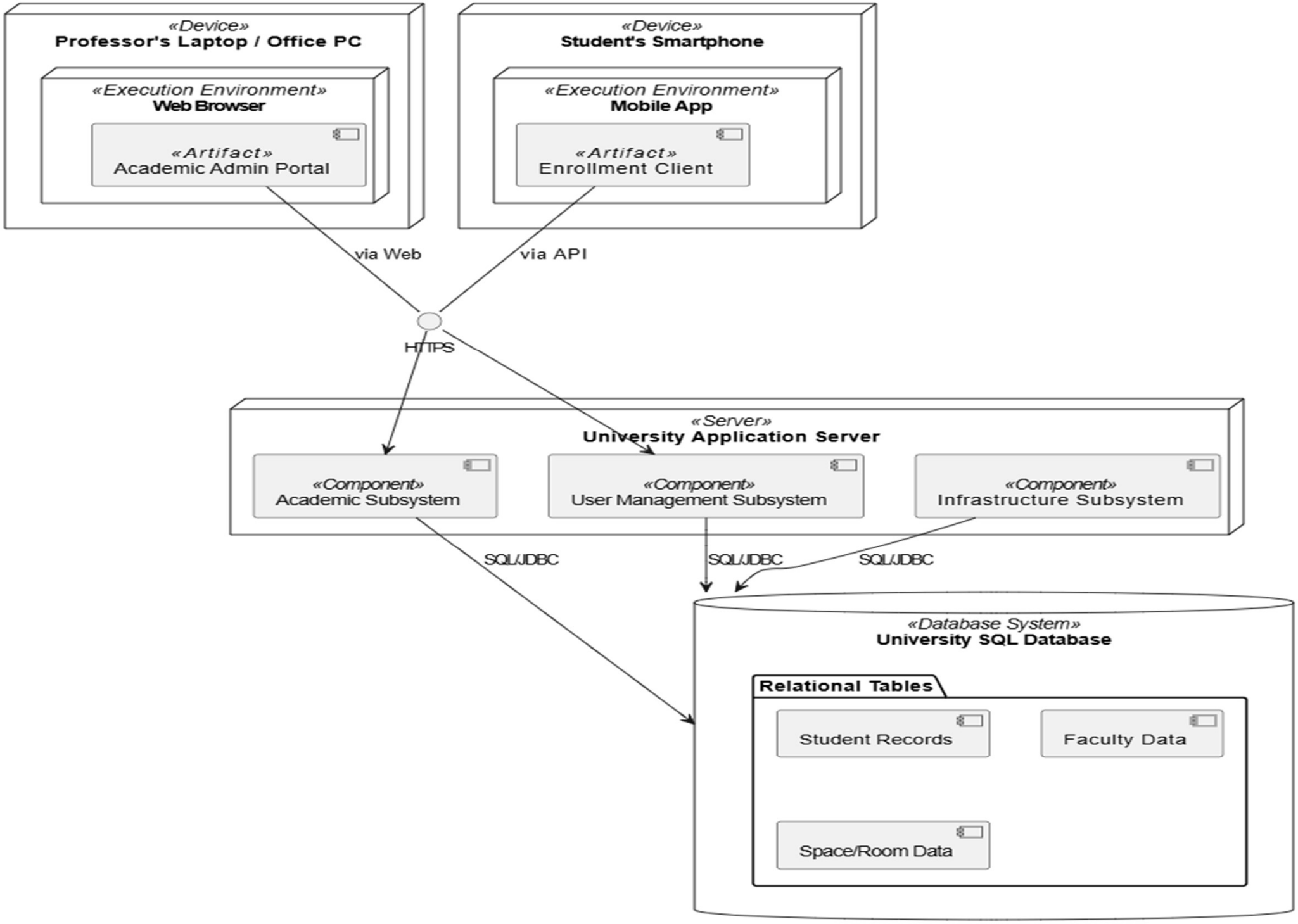
Phase 1: System Design - Subsystem Decomposition



Step 2: Hardware & Data Mapping

- **Task:** Decide where this data lives.
  - Will the Student records be stored in a Relational Database (SQL) or a simple File?
  - Will the Professor access the system via a Mobile App or a Web Browser?
- **Skill:** Understanding **Persistence** and **Deployment**.

Phase 1: Step 2 - Hardware & Data Mapping



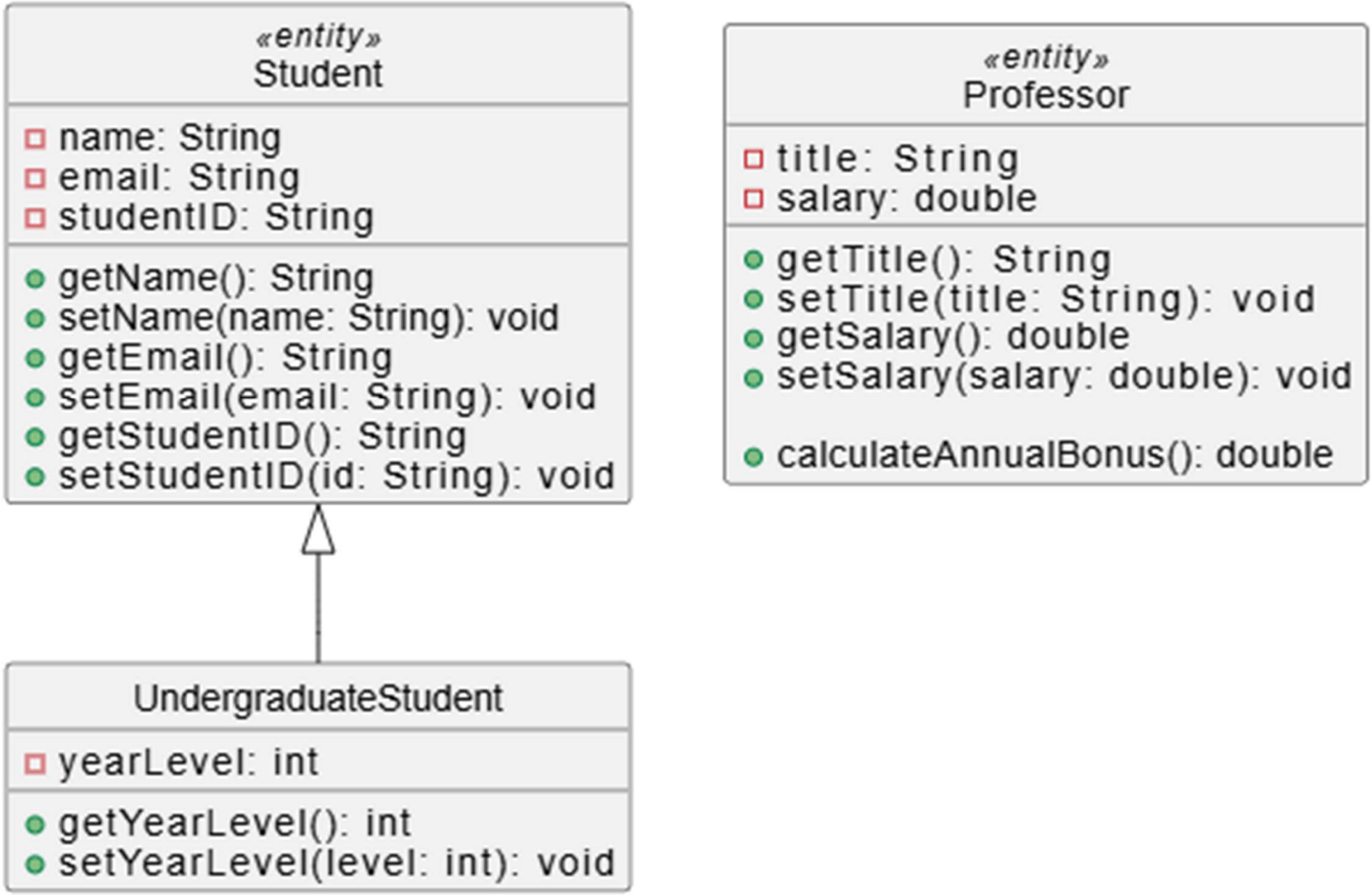
Phase 3: Object Design (Interface & Detail)

Focus: Preparing the classes for a programmer.

Step 3: Visibility and Data Types

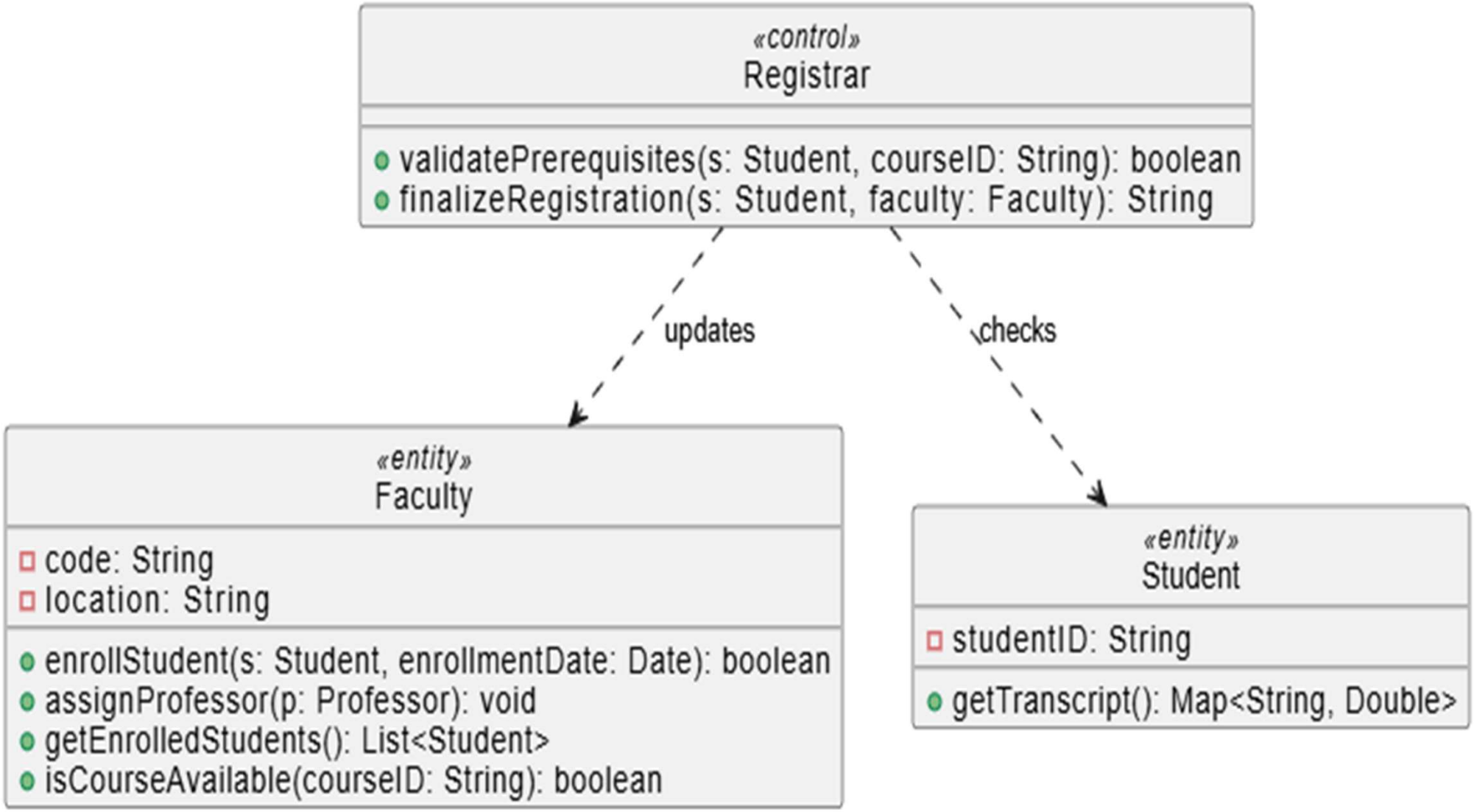
- Task: Take the Professor and Student classes.
  - Change all attributes to Private (-).
  - Add specific Data Types (e.g., salary: double, email: String).
  - Add Getters and Setters for every attribute.
- Skill: Encapsulation. you learn that "Solution Domain" objects protect their data.

Phase 3: Step 3 - Object Design (Encapsulation)



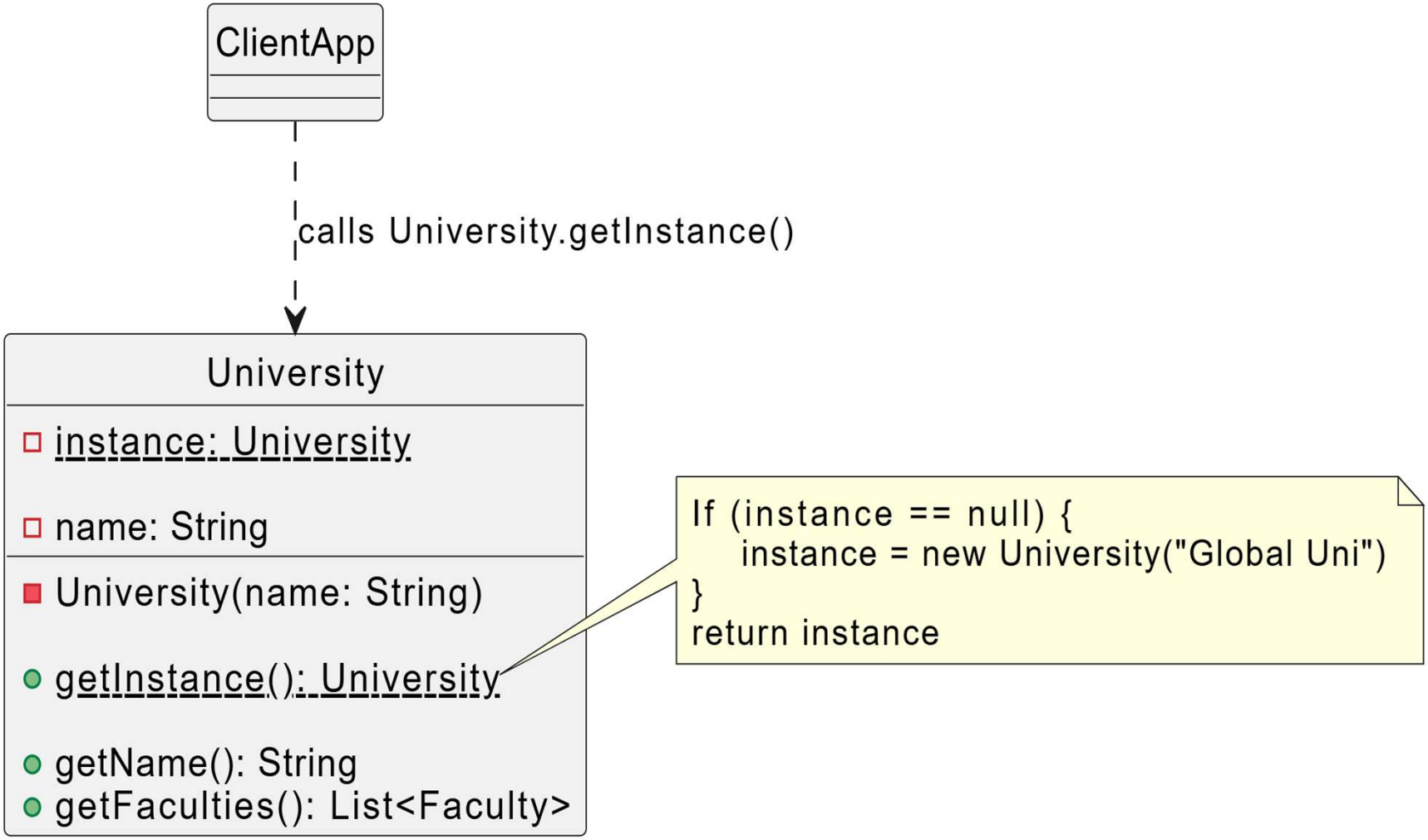
- **Step 4: Defining Method Signatures**
- **Task:** In Analysis, we said a Faculty "enrolls" a Student. Now, write the exact **Method Signature**.
  - *Example:* + enrollStudent(s: Student, date: Date): boolean
- **Skill: Interface Specification**. you must decide: What info does the method need (parameters)? What does it give back (return type)?

Phase 3: Step 4 - Method Signatures (Interface Specification)



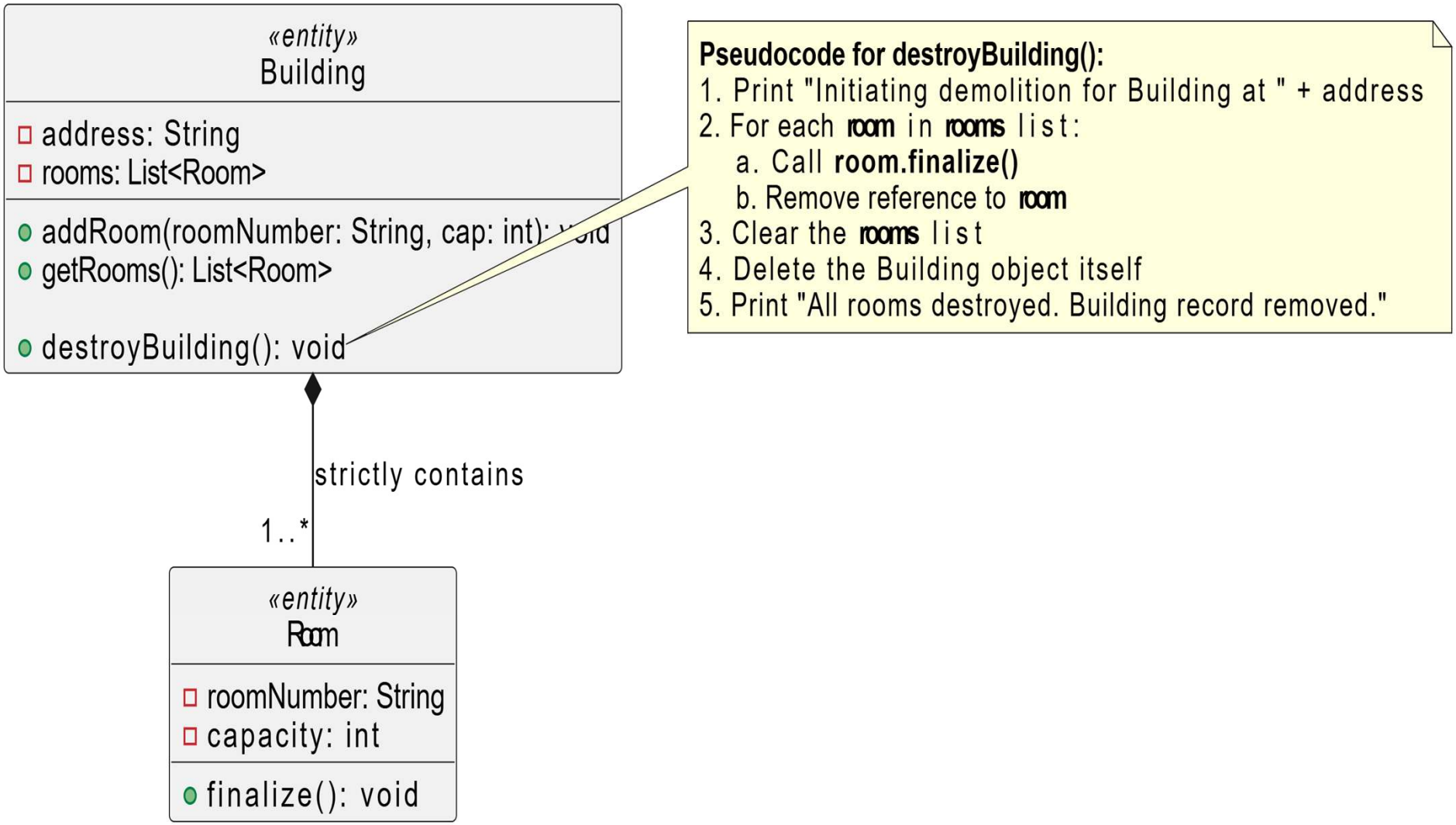
- **Phase 4: Design Patterns (Optimization)**
- *Focus: Using professional templates to solve problems.*
- **Step 5: Applying the Singleton Pattern**
- **Task:** Every university system should only have **one** University object active in memory.
  - Redesign the University class to use the **Singleton Pattern**.
  - *Requirement:* Make the constructor private and add a static getInstance() method.
- **Skill: Design Pattern Application**.

Phase 4: Step 5 - Design Patterns (Singleton University)



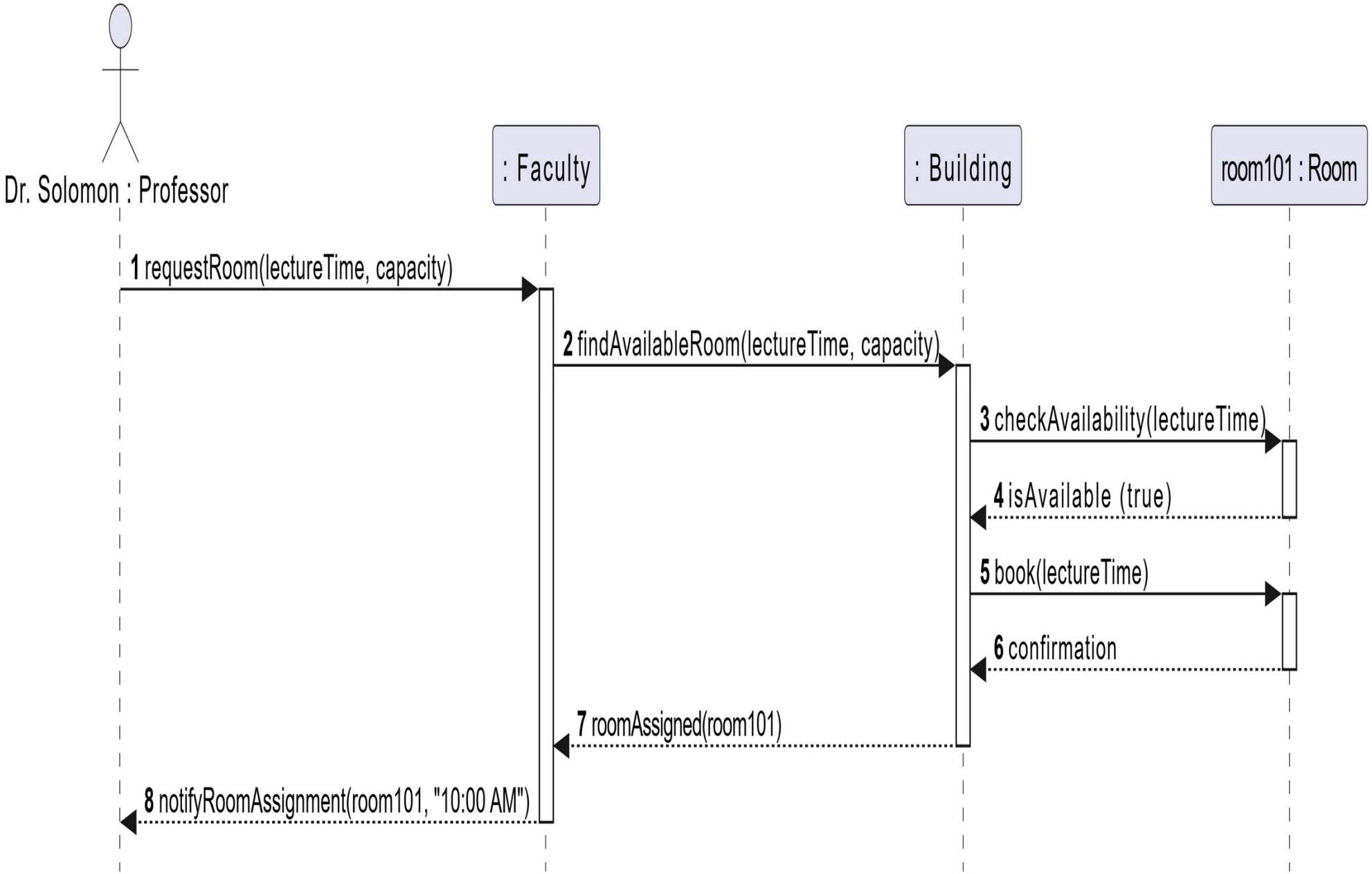
- **Step 6: Handling Composition in Code**
- **Task:** Look at the Building -- Room relationship.
  - In the Building class, add a method + destroyBuilding().
  - Write the **Pseudocode** for this method. It must ensure that all Room objects associated with it are also deleted.
- **Skill: Lifecycle Management**.

Phase 4: Step 6 - Lifecycle Management (Composition)



- Phase 5: Documentation (The ODD)
- Focus: Communicating the design.
- Step 7: The Sequence Diagram (Logic Flow)
- Task: Draw a Sequence Diagram for the scenario: "A Professor requests a Room for a lecture.“
- Show the call from Professor ➔Faculty ➔Building ➔Room.
- Skill: Dynamic Modeling. This proves the "Static" class diagram actually works to solve a "Dynamic" process.

Phase 5: Step 7 - Sequence Diagram (Request Room)



End of ch-6